

opcode_processor — DUT Specification

1. Overview

opcode_processor is a synchronous block with a 4-bit opcode input and an 8-bit data input. On each rising clock edge, while not in reset, it decodes opcode, applies the selected operation to data, and drives an 8-bit result. Opcodes fall into three functional groups (load/store, ALU, and misc), plus reserved and illegal codes that must not be exercised.

2. Interface

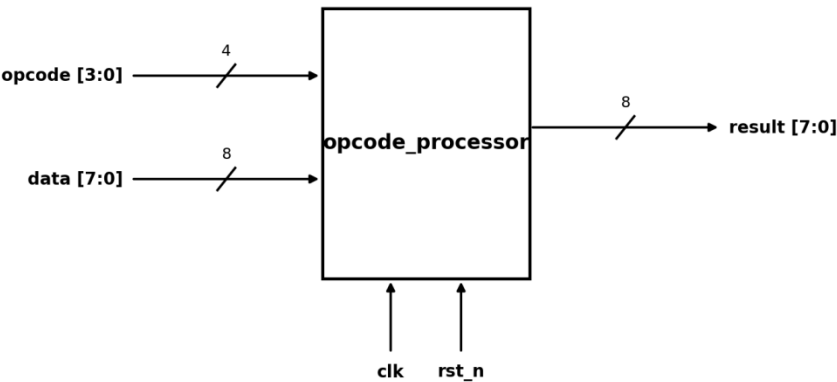


Figure 1: opcode_processor Interface Architecture

Port	Dir	Width	Description
clk	input	1	System clock. Rising-edge active (drives a 4 ns period).
rst_n	input	1	Active-low reset. TB holds low then releases before streaming stimulus.
opcode	input	4	Operation selector. Chooses the function applied to data.
data	input	8	Operand byte fed to the selected operation.
result	output	8	Processed output byte for the current opcode/data.

Parameters: none. The datapath is fixed at a 4-bit opcode and an 8-bit data/result byte.

3. Opcode map

opcode	Class	Meaning / notes
1	load_store	LOAD. A LOAD is always followed by a STORE in the stimulus.
2	load_store	STORE.
4, 5, 6	alu_ops	Arithmetic / logic operations on data.
0, 3, 7, 8	misc_ops	Miscellaneous / control operations.
9 - 14	reserved	Reserved. Excluded from coverage leave unused.
15	illegal	Illegal opcode. Must never be driven.

4. Behavior and timing

- Reset:** rst_n is asserted low, held, then released before any stimulus; the block starts from a known state after release.
- Stimulus timing:** one new (opcode, data) pair is applied per clock, aligned to the negedge, so inputs are stable across the following rising edge that the DUT samples.
- Ordering rule:** the stimulus guarantees every LOAD (1) is immediately followed by a STORE (2); the design may rely on this load-then-store sequence.
- Scope of this TB:** the testbench is coverage-driven and does not check result against a reference, so the exact arithmetic per opcode is not defined here take result semantics from the RTL or design spec. Values above are inferred from the coverage bins and constraints.